Introducing Quail: An Environment for Agentic Qualitative Analysis

Presenters

Dashiell Coyier <coyierd@carleton.edu>, Data Scientist for the DataSquad, Carleton College, 0009-0003-1650-9370

Keywords

Agentic AI, qualitative analysis, research software

Abstract

Researchers often have more text than anyone can read closely [1]. Retrieval-augmented generation (RAG) [2] helps an AI agent with a large dataset by handing it the few entries that best match a question. This can answer “what did the news say about lines at the polls?” but not “how often, and on which networks?”

Quail lets an agent analyze a text dataset the way a researcher works in a Jupyter notebook [3]. Each cell the agent sends runs in a Python kernel that stays alive between cells and has Quail’s analysis language preloaded. Fields, the columns of a dataset, build recipes: expressions such as semantic or keyword scores give a value for each entry, and predicates built from them select entries. Recipes are kept in variables, and nothing runs until one of four functions uses them: `count` and `values` work over every match, `retrieve` returns matches to read, and `tag` saves the agent’s analysis on the entries. For the second question, a cell might run:

```
close = Field("body").semantic("waiting to vote") > 0.5
print(count(close, by=Field("network")))
```

A later cell can tag the matches that fit, as in qualitative coding, and the tag becomes a field that can be counted like any other. Because all four are Python functions, one cell can combine them and return only what matters. Agents often work better this way than by calling tools one at a time [4], an approach now commonly called “code mode” [5]. The language also gives a researcher and an agent a shared vocabulary: what the agent counts, reads or tags is defined in code, so a researcher can see exactly what it did and question it.

Each cell’s code, output and tags are saved to the session log before the agent sees the result, and the kernel blocks network, file and subprocess access. A study is an ordinary folder of text files that git can share like code. Each kernel run writes its own log file, so agents working in parallel push separate files that merge without conflicts.

Quail needs only a shell, so it runs wherever an agent does, on a laptop or a cloud VM. In one test, TV news transcripts from six U.S. presidential election years, 2004 to 2024, were prepared as a study. Asked in Slack how Obama is characterized, a Cursor cloud agent [6], started from a snapshot holding Quail and the indexed study, ran nearly 20 cells and replied 27 minutes later. One cell found “Obama is not ready” in 185 segments of 2008 coverage (CNN 70, Fox 41, MSNBC 37). Quail 0.94 is an alpha release for Linux and macOS under the Apache-2.0 license [7]. Its results have not yet been formally evaluated, which is our next step.

References

1. The DataSquad Experiment: Lessons for Preparing Data and Computer Scientists for Work. Paula Lackie, Elliot Pickens, and Dashiell Coyier. arXiv:2511.19688. 2025. https://arxiv.org/abs/2511.19688
2. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Patrick Lewis et al. Advances in Neural Information Processing Systems 33. 2020. https://arxiv.org/abs/2005.11401
3. Jupyter Notebooks – a publishing format for reproducible computational workflows. Thomas Kluyver et al. Positioning and Power in Academic Publishing: Players, Agents and Agendas. IOS Press. 2016. https://doi.org/10.3233/978-1-61499-649-1-87
4. Executable Code Actions Elicit Better LLM Agents. Xingyao Wang et al. Proceedings of the 41st International Conference on Machine Learning, PMLR 235. 2024. https://arxiv.org/abs/2402.01030
5. Code Mode: the better way to use MCP. Kenton Varda and Sunil Pai. Cloudflare Blog. September 26, 2025. https://blog.cloudflare.com/code-mode/
6. Cursor agents can now control their own computers. Jonas Nelle and Alexi Robbins. Cursor. February 24, 2026. https://cursor.com/blog/agent-computer-use
7. Quail. Dashiell Coyier. GitHub repository. 2026. https://github.com/dcoyier/Quail

Connection to Mission, Goals, & Interests of US-RSE Community

This poster connects to US-RSE's Resources and Advocacy goals. For Resources, Quail is an open-source reference, under the Apache-2.0 license, for research software engineers building AI-assisted tools whose results researchers need to be able to check. The design decisions it presents come up in other projects as well: giving an agent a Python kernel with a small analysis language instead of many separate tool calls, so that a researcher can see exactly what it did; keeping the source data read-only; saving each cell to a log before the agent sees its result; and storing a study as plain files that git can share and review. The poster presents these decisions so that they are useful to people building their own tools as well as to people who might use Quail.

For Advocacy, Quail is an example of what a campus data and software support group can build in response to a local need. It came out of the DataSquad, a student-staffed group at Carleton College, after we saw campus offices collect open-ended survey answers they did not have time to analyze. It also addresses this year's theme, Advancing Science in the Age of AI, for qualitative research, where researchers need to see exactly what an agent did, and question it, before they can trust its analysis.

AI Acknowledgement

OpenAI ChatGPT (GPT-5.6 Thinking) was used during the revision of the submitted abstract. For this camera-ready version, Anthropic Claude (Claude Opus 5.5, used through a Cursor cloud agent) drafted the revised Abstract, References, and Connection sections from the submitted abstract and the Quail documentation. Anthropic Claude, used through Claude Code, then revised these sections to match the poster. The author reviewed and verified all content and takes responsibility for the submitted content.